

LarShield Database Audit Report

Schema Inspection, Policy Consent Analysis & Production Enhancements

Executive Summary: An in-depth structural inspection of local_schema.sql (PostgreSQL 18 dump from live environment) was conducted. The schema contains **13 core database tables** handling organizations, user authorization, vulnerability scan results, payment logs, and scheduled tasks. While the current SQL dump completely matches the application SQLAlchemy ORM models, critical gaps were identified regarding **User Policy Consent Tracking**, missing **Foreign Key Performance Indexes**, and **Vulnerability Lifecycle Status Tracking**.

1. Policy & Terms Consent Acceptance Audit

Question Answered: Does the database current save when a user reads and agrees to the Terms of Service & Privacy Policy?

Audit Finding: **NO (Missing in local_schema.sql).**

In the current users table, user records only store standard authentication fields: id, email, password_hash, first_name, last_name, role, org_id, failed_login_attempts, locked_until, and created_at.

There is **no timestamp or consent flag** recording when a user accepted the Legal Terms, Privacy Policy, or Acceptable Use Policy during registration.

Recommended Fix for Policy Consent Tracking:

```
ALTER TABLE public.users
  ADD COLUMN terms_accepted_at TIMESTAMP WITHOUT TIME ZONE,
  ADD COLUMN privacy_policy_agreed_at TIMESTAMP WITHOUT TIME ZONE,
  ADD COLUMN policy_version_agreed VARCHAR(20) DEFAULT 'v1.0';
```

2. Comprehensive Breakdown of Existing Tables (13 Tables)

The following 13 tables are defined in local_schema.sql and mapped to backend_structured/models.py:

Table Name	Primary Key	Foreign Keys	Purpose & Description
organizations	id (UUID)	None	Tenant accounts, subscription tier, API keys, webhook URLs, report branding logo.
users	id (UUID)	org_id → organizations	User accounts, credentials (bcrypt hash), system roles, and login brute-force lockout status.
roles	id (UUID)	None	Predefined system authorization roles (super_admin, org_admin, soc_analyst, etc.).
subscription_tiers	id (String)	None	Plan definitions (Free, Pro, Enterprise) with monthly and yearly pricing in USD/INR.
scans	id (UUID)	org_id, user_id	Scan jobs (Target URL, scan_type, status, security score 0-100, auth headers, options, SSL info).
vulnerabilities	id (UUID)	scan_id → scans	Detailed findings per scan (Severity, CVSS, Evidence, Payload, Remediation, CWE, OWASP).
scheduled_scans	id (UUID)	org_id, user_id	Automated recurring scan schedule configurations (Daily, Weekly, Monthly cron).

alert_settings	id (UUID)	user_id → users	Per-user security alert notification rules (Email, Webhook, Severity threshold).
organization_scan_quotas	id (UUID)	org_id → organizations	Tracks scan quota allocation vs consumed count per scan type per organization.
payments	id (UUID)	org_id, user_id	Transaction logs for Stripe and Razorpay subscription payments.
audit_logs	id (UUID)	admin_id → users	Security & Administrative action audit trail.
demo_bookings	id (UUID)	None	Public landing page demo requests and meeting scheduling logs.
email_logs	id (UUID)	None	System outbound notification email logs and delivery error details.

3. Missing Foreign Key Performance Indexes

In PostgreSQL, defining a FOREIGN KEY constraint does **NOT** automatically create an index on the referencing column. Without explicit indexes, queries filtering or joining by Foreign Keys (e.g., retrieving vulnerabilities for a scan, or fetching scans for an org) will force PostgreSQL into costly sequential table scans as the database grows.

Recommended Index DDL Script:

```
CREATE INDEX idx_scans_org_id ON public.scans(org_id);
CREATE INDEX idx_scans_user_id ON public.scans(user_id);
CREATE INDEX idx_scans_status ON public.scans(status);

CREATE INDEX idx_vulnerabilities_scan_id ON public.vulnerabilities(scan_id);
CREATE INDEX idx_vulnerabilities_severity ON public.vulnerabilities(severity);

CREATE INDEX idx_users_org_id ON public.users(org_id);
CREATE INDEX idx_payments_org_id ON public.payments(org_id);
CREATE INDEX idx_payments_user_id ON public.payments(user_id);
CREATE INDEX idx_scheduled_scans_is_active ON public.scheduled_scans(is_active);
```

4. Recommended Schema Additions & Modernization

To support enterprise SaaS requirements, the following column and table additions are recommended:

Target Area	Proposed Addition	Business & Technical Value
Vulnerability Lifecycle	vulnerabilities.status vulnerabilities.remmediated_at vulnerabilities.remmediated_by	Allows security teams to track resolution state (Open, In Progress, Resolved, Ignored) and record resolution audit timestamps.
Two-Factor Auth (2FA)	users.mfa_enabled users.mfa_secret	Enables TOTP-based Multi-Factor Authentication for enhanced enterprise user account security.
Scan Diagnostics	scans.duration_seconds scans.error_message scans.critical_count	Captures execution performance metrics, exact crash tracebacks on failure, and pre-aggregated vulnerability counts for fast dashboard rendering.
Report Management	New table: public.reports	Stores generated PDF/CSV report history, download links, and generation metadata per organization.

5. Complete Master Migration SQL Script

You can execute the following combined SQL migration script on your PostgreSQL database to apply all enhancements:

```
-- 1. Policy & Terms Consent Fields
ALTER TABLE public.users
  ADD COLUMN IF NOT EXISTS terms_accepted_at TIMESTAMP WITHOUT TIME ZONE,
  ADD COLUMN IF NOT EXISTS privacy_policy_agreed_at TIMESTAMP WITHOUT TIME ZONE,
  ADD COLUMN IF NOT EXISTS mfa_enabled BOOLEAN DEFAULT FALSE,
  ADD COLUMN IF NOT EXISTS mfa_secret VARCHAR(100);

-- 2. Vulnerability Lifecycle Tracking
ALTER TABLE public.vulnerabilities
  ADD COLUMN IF NOT EXISTS status VARCHAR(50) DEFAULT 'open',
  ADD COLUMN IF NOT EXISTS remediated_at TIMESTAMP WITHOUT TIME ZONE,
  ADD COLUMN IF NOT EXISTS remediated_by VARCHAR(36);

-- 3. Scan Metrics & Performance Summary
ALTER TABLE public.scans
  ADD COLUMN IF NOT EXISTS duration_seconds INTEGER,
  ADD COLUMN IF NOT EXISTS error_message TEXT,
  ADD COLUMN IF NOT EXISTS critical_count INTEGER DEFAULT 0,
  ADD COLUMN IF NOT EXISTS high_count INTEGER DEFAULT 0;

-- 4. Reports Table
CREATE TABLE IF NOT EXISTS public.reports (
  id VARCHAR(36) PRIMARY KEY,
  scan_id VARCHAR(36) NOT NULL REFERENCES public.scans(id) ON DELETE CASCADE,
  org_id VARCHAR(36) NOT NULL REFERENCES public.organizations(id) ON DELETE CASCADE,
  report_type VARCHAR(50) NOT NULL,
  file_path VARCHAR(500),
  generated_at TIMESTAMP WITHOUT TIME ZONE DEFAULT NOW()
);

-- 5. Create Missing Performance Indexes
CREATE INDEX IF NOT EXISTS idx_scans_org_id ON public.scans(org_id);
CREATE INDEX IF NOT EXISTS idx_vulnerabilities_scan_id ON public.vulnerabilities(scan_id);
CREATE INDEX IF NOT EXISTS idx_users_org_id ON public.users(org_id);
```